CyClaw

Offline-First, RAG-Enforced Soul Agent
Architecture Reference Document | v1.4.0 | June 2026

Chris Grady | @CGFixIT | github.com/CGFixIT/CyClaw | cgfixit.com

v1.4.0 Update Highlights
- **Python 3.12 hardened** — requirements.txt + constraints.txt validated against Python 3.12 runtime
- **CVE patches** — CVE-2025-62727, CVE-2026-48710, CVE-2026-28277, CVE-2026-44843 addressed via dependency updates
- **Reproducible builds** — constraints.txt pins full transitive dependency tree
- **Expanded test suite** — test_mcp_server.py, test_security.py, test_telemetry_kill.py, test_conftest_fixtures.py added
- **New documentation** — docs/CODE_SECURITY_REVIEW_2026-06-16.md, docs/SETUP.md, docs/VERIFICATION_REPORT_3.12.md
- **/soul/restore** endpoint — restores soul.md from .bak backup
- All 5 security invariants preserved and structurally strengthened

Property	Value
Version	1.4.0 (requirements audit + Python 3.12 hardening + CVE patches)
Runtime	FastAPI + LangGraph + LM Studio (local GGUF)
Retrieval	ChromaDB (vector) + BM25 (keyword) + RRF fusion
Embeddings	sentence-transformers / all-MiniLM-L6-v2 (CPU, 384-dim)
Soul Governance	SQLite + Markdown, versioned, atomic writes, drift-detected, 365-day TTL
Rate Limiting	In-memory sliding window, 60 req/min per IP, HTTP 429 on exceed
Sanitizer	13 OWASP-aligned banned patterns + corpus-time stripping
Fallback	Grok via xAI API (user-gated, config-gated, env-gated)
MCP Server	Retrieval-only, sampling = null, no LLM access
Binding	127.0.0.1:8787 (offline-first, CORS-restricted)
Audit	SHA-256 hashed, PII-redacted, append-only JSONL
Browser UI	terminal.html (Soul Console) + extractor.html

Version History

Version	Status	Notes
v1.2.0	Superseded	Baseline production release
v1.3.0	Pre-Langgrinch	Rate limiting (60/min), 13 OWASP patterns, soul SHA-256 drift detection, atomic writes, TTL→365 days
v1.4.0	Production (current)	Updated requirements.txt to patch vulns and modernize for Python 3.12. Added constraints.txt for reproducible builds. Expanded test suite.
v1.5.0	Planning	Fix Stemmer.py, SQL write placeholder cleanup, Dropbox corpus sync integration, BM25 SHA integrity detection on load

Table of Contents

A. System Overview

CyClaw is a six-layer architecture where each layer has a single responsibility and explicit boundaries. The LangGraph controller is the security enforcement mechanism — routing decisions are made by graph topology, not by prompt instructions. Version 1.4.0 preserves the v1.3.0 rate limiter, CORS middleware, and StaticFiles mount, and hardens the dependency layer via constraints.txt and CVE-patched packages.

Client Interfaces

Interface	Protocol	Entry Point
CLI / Scripts	HTTP	POST /query, /health, /soul/*
Browser UI (terminal.html)	HTTP	/ (StaticFiles) → /query, /soul/*
Extractor (extractor.html)	HTTP	Corpus extraction tool
LM Studio / Claude Desktop	MCP (JSON-RPC / stdio)	mcp_hybrid_server.py

Layer Stack

Layer	Module(s)	Responsibility
A. Gateway	gate.py, mcp_hybrid_server.py	Rate limit, sanitization, CORS, static serving, GraphState init
B. Controller	graph.py	7-node LangGraph: retrieve → route → respond → audit
C. Retrieval	retrieval/*.py	Corpus indexing, hybrid search (Chroma + BM25 + RRF)
D. Soul	utils/personality.py, data/personality/	Versioned identity, drift-detected, atomic writes, TTL pruning
E. Models	llm/client.py	LM Studio (local), sentence-transformers, Grok (triple-gated)
F. MCP Server	mcp_hybrid_server.py	Retrieval-only tool; sampling = null

B. Gateway Layer

The FastAPI gateway (gate.py) and MCP hybrid server are the only external interfaces. Both bind to 127.0.0.1 — no external network exposure by default. Rate limiting is hardcoded in gate.py as a 60 req/min per-IP sliding window. **Note:** security.rate_limit.enabled in config.yaml is documented but config-driven rate limiting is not yet implemented — rate limiting is enforced in code, not config.

HTTP Endpoints

Endpoint	Method	Description
/query	POST	Main entry — rate-limit, sanitize, build GraphState, invoke graph
/health	GET	Liveness + dependency health checks (LM Studio, Grok, embeddings)
/soul	GET	Current soul content, version number, source path
/soul/propose	POST	Diff preview with SHA hashes — does NOT apply
/soul/apply	POST	Applies evolution — requires human reason string
/soul/reload	POST	Reload soul.md from disk after manual edit
/soul/restore	POST	Restores soul.md from .bak backup (NEW in v1.4.0)
/	GET	Serves terminal.html (Soul Console) via StaticFiles mount

Rate Limiter (thread-safe in-memory)

```
_rate_limits = defaultdict(list)
RATE_LIMIT_REQUESTS = 60
RATE_LIMIT_WINDOW = 60  # seconds

def check_rate_limit(client_ip: str) -> bool:
    now = time.time()
    _rate_limits[client_ip] = [t for t in _rate_limits[client_ip] if now - t < RATE_LIMIT_WINDOW]
    if len(_rate_limits[client_ip]) >= RATE_LIMIT_REQUESTS:
        return False
    _rate_limits[client_ip].append(now)
    return True
```

Gateway Invariants

- Rate-limit check runs FIRST (per-IP, 60/min sliding window) — exhausted clients hit 429 before any work
- Banned-pattern prompt filter (13 patterns) runs before any LLM call
- Max input size 4000 chars enforced at gateway level
- CORS restricted to security.allowed_origins (config-driven allowlist)
- GraphState initialized with: query, user_confirmed_online, soul_core
- All responses follow uniform JSON schema (answer, sources, model_used, retrieval_mode, needs_confirm, error)
- Structured error handling with typed exceptions (RAGError → PromptInjectionError, IndexNotFoundError, LLMServiceError, etc.)
- Telemetry kill-switch env vars set at module top, BEFORE any langchain/chromadb import

C. LangGraph Controller

The controller is a 7-node directed graph where **topology IS the security policy**. Routing decisions are structural — enforced by graph edges, not by prompt wording. Every execution path terminates at the audit_logger node before returning to the gateway. Soul injection happens at the prompt construction level inside local_llm and offline_best — there is no soul graph node. Evolution is an explicit HTTP endpoint, not a graph operation, per model council consensus: no autonomous self-modification in the graph.

Graph Topology (7-node directed)

```
[ENTRY]
  ↓
1. retrieve (Chroma + BM25 + RRF fusion → retrieved_docs, top_score)
  ↓
2. route_score (top_score >= cfg.retrieval.min_score [0.028 RRF]?)
  ■■ score >= threshold ■■→ 3. local_llm (soul + query + docs → LM Studio)
  ■■ score < threshold  ■■→ 4. user_gate (sets needs_user_confirm; awaits resubmit)
                              ■■ user=yes + hybrid + grok.enabled ■■→ 5. grok_fallback
                              ■■ user=no OR Grok disabled          ■■→ 6. offline_best
  ↓
7. audit_logger (SHA-256 query hash + PII redact → logs/audit.jsonl) → END
```

All paths converge at audit_logger. The convergence is enforced by graph edges, not by code discipline — there is no shortcut path. Personality interactions are also recorded to the shadow DB after audit so soul-level history stays consistent with the audit trail.

Node Descriptions

Node	Inputs	Outputs	Notes
retrieve	query	retrieved_docs, top_score, retrieval_mode	Always first; degrades gracefully
route_by_score	top_score	Conditional edge	Graph topology, not LLM decision
local_llm	soul_core, query, retrieved_docs	answer, answer_model, answer_sources	LM Studio; soul injected as preamble
user_gate	top_score (below threshold)	needs_user_confirm=True	Pauses; client must resubmit
grok_fallback	user_confirmed_online=True + gates	answer, answer_model	Triple-gated; no local context forwarded by default
offline_best	Any non-Grok path	answer, answer_model	Best-effort local; soul-injected
audit_logger	Full GraphState	audit_event appended to JSONL	SHA-256 hash + PII redaction; always terminal

GraphState Schema

Field	Type	Set By
query	str	Gateway (from user input)
user_confirmed_online	bool | None	Gateway (from client resubmit)
retrieved_docs	list[RetrievedDoc]	retrieve node
top_score	float	retrieve node
retrieval_mode	str	retrieve node (hybrid|vector|bm25|none)
needs_user_confirm	bool	route_by_score / user_gate
answer	str	local_llm | grok_fallback | offline_best
answer_model	str	Responding node (local|grok|offline-best-effort)
answer_sources	list[RetrievedDoc]	Responding node
audit_event	dict	audit_logger node
error	str | None	Any node on failure

D. Retrieval Layer

Retrieval is the unconditional first node in the graph. The hybrid retriever combines semantic (ChromaDB cosine HNSW) and keyword (BM25Okapi) search via Reciprocal Rank Fusion (RRF). If either path fails, the retriever degrades gracefully to whichever path remains viable.

Indexing Pipeline (offline batch)

- **load_corpus()** — recursive read of .md/.txt from data/corpus/
- **chunk_document(size=512, overlap=50)** with sentence-boundary snapping
- **sanitize_chunk()** — strips banned patterns at ingestion (13 patterns in v1.3+)
- **tokenize_and_stem()** — enhanced Porter stemmer (technical-vocab safe)
- **embed_chunks()** — 384-dim via all-MiniLM-L6-v2 (CPU, batch=32)
- **Write Chroma collection** → index/chroma_db/ (cosine HNSW)
- **Build BM25 index** → index/bm25.pkl (pickle: bm25 + chunks + metadata)

Query-Time Hybrid Search

Component	Method	Effective Weight
Semantic	Chroma vector search (embed → cosine sim)	1.0 (equal)
Keyword	BM25 (tokenize + stem → term frequency)	1.0 (equal)
Fusion	RRF (k=60): score = Σ 1/(k + rank_i)	—

Each SearchResult carries full RRF provenance: semantic_score, semantic_rank, keyword_score, keyword_rank, rrf_score, rrf_semantic_contrib, rrf_keyword_contrib. Fields are exposed in the SourceInfo Pydantic response model. **Design note:** CyClaw uses equal-weighted RRF. The retrieval.hybrid.rrf config keys vector_weight: 0.6 / bm25_weight: 0.4 are documentation-only placeholders and are not multiplied into the RRF contribution by the current hybrid_search.py. To opt into true weighted RRF, multiply contrib *= weight in both loops AND retune retrieval.min_score.

Retrieval Configuration

Parameter	Value	Notes
top_k_semantic	5	Max vector results
top_k_keyword	5	Max BM25 results
rrf_k	60	RRF smoothing constant
max_context_tokens	5000	Token budget for retrieved context
min_score	0.028	RRF fused-rank threshold (NOT cosine sim — different scale)
chunk_size	512	Chars per chunk (sentence-snapped)
chunk_overlap	50	Overlap between adjacent chunks
batch_size	50	Embedding batch size (config); inference batch is 32

E. Soul Governance

Identity ≠ memory ≠ topology. Soul is who the agent IS. Memory is what the agent KNOWS. Topology is what the agent CAN DO. Each is governed independently. v1.3.0 hardened the soul layer in three concrete ways while preserving the original write-order invariant and threading.Lock serialization. **v1.4.0 preserves all soul governance as-is; no structural changes to this layer.**

v1.3.0 Hardening (retained in v1.4.0)

Drift Detection. On every PersonalityManager.__init__, the SHA-256 of soul.md is compared to the latest sha256 in soul_versions. Mismatch triggers a forensic audit_log event AND inserts an automatic recovery version row so the audit trail always reflects true on-disk state.

Atomic Evolution. apply_evolution writes to a soul.md.tmp sibling file then calls os.replace() for a POSIX-atomic rename. Eliminates partial-write corruption risk on crash or power loss.

Automatic TTL Pruning. maintenance(ttl_days) is invoked from __init__ using personality.interaction_ttl_days (default 365) so old interaction rows are cleaned on every gateway start.

Threading Lock Preserved. threading.Lock still serializes every SQLite write. Drift recovery, version insert, interaction insert, and TTL prune all hold the lock.

Write-Order Invariant (Crash-Safe)

```
1. Write proposed soul to soul.md.tmp
2. os.replace(soul.md.tmp, soul.md)   # POSIX-atomic rename
3. INSERT new version row into SQLite  (under threading.Lock)
4. Update in-memory soul_core          (runtime matches persisted state)
5. audit_log({'event': 'soul_evolution_applied', ...})
```

Any crash between steps leaves the system recoverable: soul.md is either the old or new content (never partial) and the SQLite version log records the evolution intent.

Soul API Constraints

- **GET /soul** — read-only (current content + version + path)
- **POST /soul/propose** — generates (current_sha, proposed_sha) diff WITHOUT applying
- **POST /soul/apply** — requires human-authored reason string; no anonymous mutations
- **POST /soul/reload** — re-reads soul.md after manual edit
- **POST /soul/restore** — restores soul.md from .bak backup (NEW in v1.4.0)
- **No endpoint allows silent or automated soul rewrites**

Personality Config

Parameter	Value	Description
personality.enabled	true	Toggle soul injection
personality.soul_path	data/personality/soul.md	File-as-truth
personality.db_path	data/personality/cyclaw_soul.db	Shadow DB
personality.interaction_ttl_days	365 (was 90 in v1.2)	Prune old interaction logs

F. Model Layer

Model	Type	Endpoint	Gating
all-MiniLM-L6-v2	Embeddings (384d)	CPU local, .emb_cache/	Always available
Qwen 2.5 7B (GGUF)	Chat LLM	127.0.0.1:1234/v1	LM Studio required
grok-beta	Chat LLM (external)	api.x.ai/v1	Triple-gated (see below)

LocalLLMClient: Timeout 720s (long-context inference), temperature 0.3, max_tokens 5000. Defensive: all client calls raise typed LLMServiceError subclasses mapped to HTTP responses by the gateway.

GrokClient — Triple Gate (defense in depth)

1. app.mode == 'hybrid' (config flag)
2. models.grok.enabled == true (config flag)
3. user_confirmed_online == true (per-query, set by client resubmit)

All three must be simultaneously true. The Grok client validates each precondition defensively even though the graph topology already prevents invalid calls. **policy.fallback.send_local_context_to_grok = false** by default: retrieved docs are NOT forwarded to Grok unless the user explicitly opts in.

G. MCP Server

Property	Value
Protocol	JSON-RPC over stdio
Server Name	cyclaw-hybrid-rag v1.0.0
Tool	hybrid_search
Input schema	{ query: string, top_k?: int, mode?: 'hybrid'|'vector'|'bm25' }
Sampling	null — cannot invoke any LLM (retrieval-only by protocol design)
Write access	None (read-only against existing indices)

The sampling = null constraint is structural, not policy. The MCP server cannot autonomously invoke an LLM. Any LLM processing must go through the gateway's LangGraph controller with full audit coverage — the MCP path bypasses none of the five security invariants.

H. Security and Sanitization

v1.3.0 expanded the prompt-injection sanitizer from 8 to 13 OWASP-aligned banned patterns. v1.4.0 hardens the dependency layer with CVE patches and pins the full transitive tree via constraints.txt. The sanitizer operates at two levels: user input (reject on match, raises PromptInjectionError) and corpus chunks at ingestion (strip + replace with [FILTERED]). Patterns are config-driven and hot-reloadable.

Banned Patterns (13, OWASP-aligned)

Pattern	OWASP Category	Status
ignore previous instructions	LLM01 — Prompt Injection	v1.2 (8 base)
ignore all instructions	LLM01 — Prompt Injection	v1.2 (8 base)
you are now in developer mode	LLM01 — Prompt Injection	v1.2 (8 base)
as an ai with no restrictions	LLM01 — Jailbreak	v1.2 (8 base)
system prompt:	LLM01 — Prompt Extraction	v1.2 (8 base)
disregard your instructions	LLM01 — Prompt Injection	v1.2 (8 base)
pretend you are	LLM01 — Role Hijacking	v1.2 (8 base)
jailbreak	LLM01 — Jailbreak	v1.2 (8 base)
do anything now	LLM01 — Prompt Injection	NEW in v1.3
act as uncensored	LLM01 — Jailbreak	NEW in v1.3
bypass safety	LLM01 — Prompt Injection	NEW in v1.3
DAN	LLM01 — Role Hijacking	NEW in v1.3
ignore safety	LLM01 — Prompt Injection	NEW in v1.3

PII Redaction (audit logger)

Type	Pattern	Replacement
Emails	Standard email regex	[REDACTED_EMAIL]
IPv4	Dot-notation	[REDACTED_IP]
AWS Keys	AKIA[0-9A-Z]{16}	[REDACTED_SECRET]
Slack Tokens	xoxb-[0-9a-zA-Z-]+	[REDACTED_SECRET]
API Keys	sk-[a-zA-Z0-9]{20,}	[REDACTED_SECRET]

Rate Limiting

Rate limiting is hardcoded in gate.py (60 req/min sliding window). Returns HTTP 429 with structured error JSON {error: 'Rate limit exceeded (60/min)', code: 'RATE_LIMIT'} and emits an audit_log event with the offending client IP for forensic review. **Note:** security.rate_limit.enabled in config.yaml is currently a documentation-only field; config-driven toggling is not yet implemented (planned v1.5.0).

I. Browser UI

Both UIs are served via the StaticFiles mount on the gateway and run fully against http://127.0.0.1:8787 — no external JS dependencies, no CDN-loaded scripts.

• **terminal.html** (~29 KB) — primary Soul Console: query input → /query, real-time answer + sources + retrieval metadata, Soul panel for load/propose/apply/reload/restore

• **extractor.html** (~18 KB) — corpus extraction utility (carried over unchanged from v1.2.0)

J. Test Suite

Original 8 pytest files + PowerShell smoke test remain. v1.3.0 added three test files for v1.3 features. v1.4.0 replaces test_endpoints_mocked.py (no longer in repo) with four new test files targeting MCP protocol, security regression, telemetry kill-switch, and shared fixture validation. All tests use mocked external deps — no live LM Studio / Chroma / Grok required.

File	Coverage
tests/conftest.py	Shared fixtures + TEST_CONFIG mirroring prod
tests/test_graph.py	All 7 LangGraph paths: local, grok, offline, gate, error
tests/test_gate.py	/query, /health, /soul/*, error responses
tests/test_personality.py	PersonalityManager: init, propose, apply, reload
tests/test_sanitizer.py	Banned patterns, max length, corpus sanitization
tests/test_audit.py	SHA-256 hashing, PII redaction, JSONL output
tests/test_hybrid_search.py	Semantic, keyword, fusion, graceful degradation
tests/test_stemmer.py	Plurals, verb forms, technical terms
tests/apipsTest.ps1	PowerShell API smoke tests
tests/test_rate_limit.py (v1.3)	Sliding window, expiry, per-IP isolation
tests/test_personality_changes.py (v1.3)	Drift detection, atomic apply, TTL prune, version ordering
tests/test_mcp_server.py (NEW v1.4)	MCP protocol validation, sampling=null invariant
tests/test_security.py (NEW v1.4)	BM25 pickle rejection, API key auth, logging setup
tests/test_telemetry_kill.py (NEW v1.4)	LangChain/Chroma/OTel env verification
tests/test_conftest_fixtures.py (NEW v1.4)	Shared mock fixtures for retriever, LLM, config
tests/TEST_SUITE_AUDIT.md	Test coverage audit and gap analysis

Pre-flight test runner: `python -m pytest tests/test_personality_changes.py tests/test_rate_limit.py -q ; python -m pytest tests/ -q`

K. Deployment & Validation

Install / Quick Start

Step 1 — Install PyTorch (CPU wheel first):

```
pip install torch==2.4.1+cpu --index-url https://download.pytorch.org/whl/cpu
```

Step 2 — Install remaining dependencies with constraints:

```
pip install -r requirements.txt -c constraints.txt
```

constraints.txt pins the full transitive dependency tree for reproducible builds. Always use -c constraints.txt for production installs.

ChromaDB migration note (v1.4.0): ChromaDB moved from 0.4.x to 1.5.x — delete index/ and rebuild with `python -m retrieval.indexer`.

Offline note: HF_HUB_OFFLINE=1 is set in cyclaw_telemetry_kill.env. Run the indexer once on a networked machine to cache all-MiniLM-L6-v2, then it runs fully offline.

v1.4.0 Upgrade Path (from v1.3.0)

1. Add constraints.txt to project root
2. Run `pip install -r requirements.txt -c constraints.txt` to update deps
3. Delete index/ directory and rebuild: `python -m retrieval.indexer` (ChromaDB 1.5.x schema change)
4. Copy env vars from cyclaw_telemetry_kill.env if not already set
5. Run pre-flight: `python -m pytest tests/test_security.py tests/test_telemetry_kill.py -q`
6. Start gateway: `uvicorn gate:app --host 127.0.0.1 --port 8787`
7. Smoke test: `curl -X POST http://127.0.0.1:8787/query -d '{"query":"hi"}' -H 'Content-Type: application/json'`
8. Verify: `curl http://127.0.0.1:8787/health` shows all deps healthy

Rollback Plan

If v1.4.0 misbehaves: stop the gateway, reinstall deps from v1.3.0 requirements.txt (without -c constraints.txt), restart. The cyclaw_soul.db schema is forward-compatible with v1.3.0 — no data loss.

L. v1.4.0 Change Log (Summary)

Dependency & Runtime
• requirements.txt and constraints.txt validated against Python 3.12
• CVE-2025-62727, CVE-2026-48710, CVE-2026-28277, CVE-2026-44843 patched via updated pins
• constraints.txt added for full transitive reproducible builds

Test Suite Expansion
• test_mcp_server.py — MCP protocol validation, sampling=null invariant
• test_security.py — BM25 pickle rejection, API key auth, logging setup
• test_telemetry_kill.py — LangChain/Chroma/OTel env verification
• test_conftest_fixtures.py — Shared mock fixtures for retriever, LLM, config
• Replaced deprecated test_endpoints_mocked.py

New Endpoints & Docs
• POST /soul/restore — restores soul.md from .bak backup
• docs/CODE_SECURITY_REVIEW_2026-06-16.md, docs/SETUP.md, docs/VERIFICATION_REPORT_3.12.md added

Notes / Future
• Rate limiting remains hardcoded (config-driven toggle planned v1.5.0)
• v1.5.0 planning items: Fix Stemmer.py, SQL write placeholder cleanup, Dropbox corpus sync, BM25 SHA integrity detection on load

M. Security Invariants (5 topology-enforced guarantees)

1. RAG-first retrieval — Retrieval node is unconditional first in every graph path. No LLM call occurs without prior retrieval attempt.

2. Topology = enforcement — Routing decisions (local vs user_gate vs grok vs offline) are made exclusively by graph edges and conditional nodes, never by LLM interpretation of instructions.

3. Identity ≠ memory ≠ topology — Soul (who), memory/knowledge (what), and graph topology (can do) are governed by independent subsystems with explicit boundaries and no cross-layer silent mutation.

4. Audit convergence — Every execution path terminates at audit_logger. There is no bypass; SHA-256 hashing + PII redaction is mandatory and structural.

5. Human-in-the-loop for evolution — Soul mutations require explicit human-authored reason string via /soul/apply. No anonymous or automated self-rewrites are possible via any endpoint or graph path.

All content derived from CyClaw v1.4.0 production codebase and architecture reference. This compact edition reduces whitespace via tighter typography, smaller spacers, denser tables, and 0.5" margins while preserving full technical accuracy. For the canonical long-form screenshot edition, see the original PDF in the repository.